

Desarrollo de Reportes y Estadísticas

Actividad N°: 27 **Fecha:** 07/07/2026 **Horario:** 14:00 a 20:00 **Tutor Empresarial:** Miguel Vivanco **Pasante:** Gabriel

1. Objetivo del día

Implementar los reportes y estadísticas agregadas que alimentan el dashboard administrativo (RF-11), usando el framework de agregación de MongoDB para calcular métricas sin traer todos los documentos al backend.

2. Estadísticas de tickets (`ticketController.getTicketStats`)

Métrica	Cálculo
<code>totalTickets</code>	<code>countDocuments()</code> sobre toda la colección activa
<code>ticketsCanjeados</code>	<code>countDocuments({ canjeado: true, ...filtros })</code>
<code>ticketsRestantes</code>	<code>totalTickets - ticketsCanjeados</code>
<code>porcentajeCanjeados</code>	$(\text{ticketsCanjeados} / \text{totalTickets}) * 100$, redondeado a 2 decimales
<code>evolucionDiaria</code>	Agregación agrupando por día (<code>\$dateToString</code>) los canjes realizados, ordenados cronológicamente
<code>ticketsPorPunto</code>	Agregación agrupando canjes por <code>puntoTrabajo</code> , ordenados de mayor a menor

Filtros soportados: `puntoTrabajo`, `fechaInicio`, `fechaFin` (sobre `fechaCanje`). Todas las consultas de esta función se ejecutan en paralelo con `Promise.all` para minimizar el tiempo total de respuesta.

3. Resumen de auditoría (`auditController.getAuditSummary`)

Métrica	Cálculo
<code>logsPorTipo</code>	Agrupación por <code>tipo</code> de operación (<code>canje</code> , <code>impresion</code> , <code>login</code> , etc.)
<code>logsPorUsuario</code>	Agrupación por <code>usuario</code> , con <code>\$lookup</code> a la colección <code>users</code> para incluir nombre y rol en el resultado, ordenado por cantidad descendente
<code>logsPorDia</code>	Agrupación por día (<code>\$dateToString</code>) de todos los eventos de auditoría

Al igual que en `getTicketStats`, las tres agregaciones se ejecutan en paralelo.

4. Ejemplo de pipeline de agregación (evolución diaria)

```
Ticket.aggregate([
  { $match: { canjeado: true, ...matchQuery } },
  { $group: {
```

```
    _id: { $dateToString: { format: "%Y-%m-%d", date: "$fechaCanje" } },
    count: { $sum: 1 }
  }},
  { $sort: { _id: 1 } }
]);
```

Este patrón se repite (con variaciones de campo de agrupación) tanto para tickets como para logs de auditoría, evitando traer todos los documentos individuales al servidor de aplicación solo para contarlos o agruparlos en memoria.

5. Diferencia entre "reportes de negocio" y "reportes de auditoría"

- **Reportes de negocio** (`getTicketStats`): responden la pregunta "*¿cómo va el evento?*" — cuántos boletos canjeados, cuántos faltan, evolución en el tiempo, distribución por punto de trabajo.
- **Reportes de auditoría** (`getAuditSummary`): responden la pregunta "*¿quién hizo qué y cuándo?*" — enfocados en control interno y trazabilidad, no en el avance operativo del evento.

Ambos se exponen como endpoints separados y restringidos a Jefe, pero se combinan visualmente en el Dashboard y en la página de Auditoría respectivamente.

6. Conclusiones del día

Quedan implementados los reportes agregados de tickets y de auditoría usando el framework de agregación de MongoDB, calculando las métricas del lado de la base de datos en vez de procesarlas en memoria en Node.js, lo cual es más eficiente para volúmenes grandes de datos.

Observaciones: Sin observaciones.